Converting Maven to Gradle

Converting Maven to Gradle

Converting a Apache Maven project to Gradle means changing the project's build system from Maven to Gradle for better flexibility and faster performance.

Maven uses the pom.xml file to manage project configuration and dependencies, whereas Gradle uses build.gradle files written in Groovy or Kotlin DSL. During conversion, all dependencies, plugins, and build configurations from Maven are transferred into Gradle format.

The main purpose of converting Maven to Gradle is to improve build speed, simplify project configuration, and support modern DevOps automation practices. Gradle provides incremental builds and better customization features, making it more efficient for large-scale projects.

Working Process

1. The existing Maven project is analyzed.
2. Dependencies and plugins from pom.xml are identified.
3. Gradle configuration files are created.
4. Maven dependencies are converted into Gradle dependencies.
5. The project is rebuilt and tested using Gradle.

Advantages

- Faster build execution
- Flexible project configuration
- Better performance with incremental builds
- Easier dependency management
- Improved CI/CD integration

Applications

- Web application development
- Selenium automation projects
- Continuous Integration and Deployment pipelines
- DevOps automation workflows

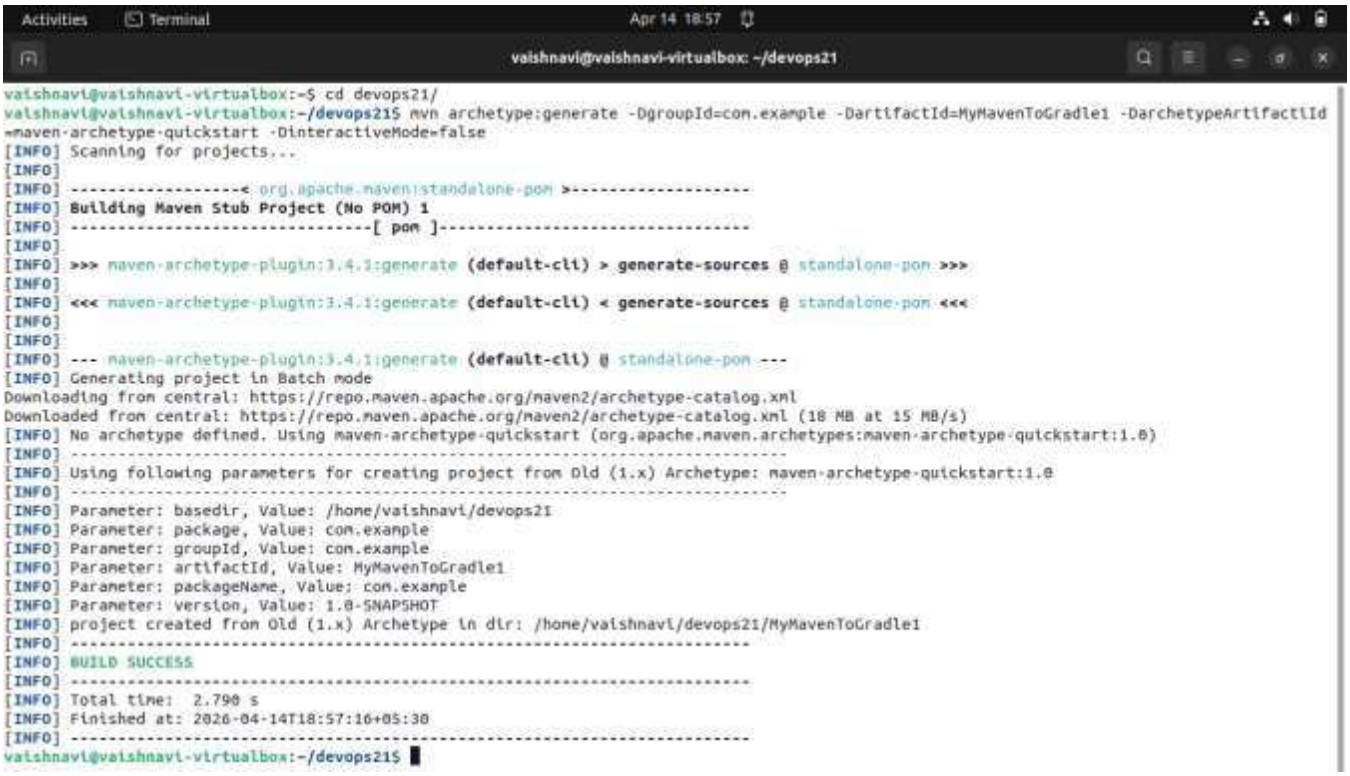
This conversion helps developers use Gradle's advanced build automation features while maintaining the existing project functionality.

Steps to execute the project:

Step 1:

Create a simple maven project with the command:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenToGradle -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false



The command can be broken down as:

DgroupId=com.example – project's base package (reverse-domain style).

-DartifactId=my-maven-app – defines the name of the project (will also be the folder name).

-DarchetypeArtifactId=maven-archetype-quickstart – sets the Maven template to be the basic Java application.

-DinteractiveMode=false – deactivates interactive mode and uses the given values automatically.

Step 2:

Edit the file pom.xml

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>MyMavenApp</artifactId>

<packaging>jar</packaging>

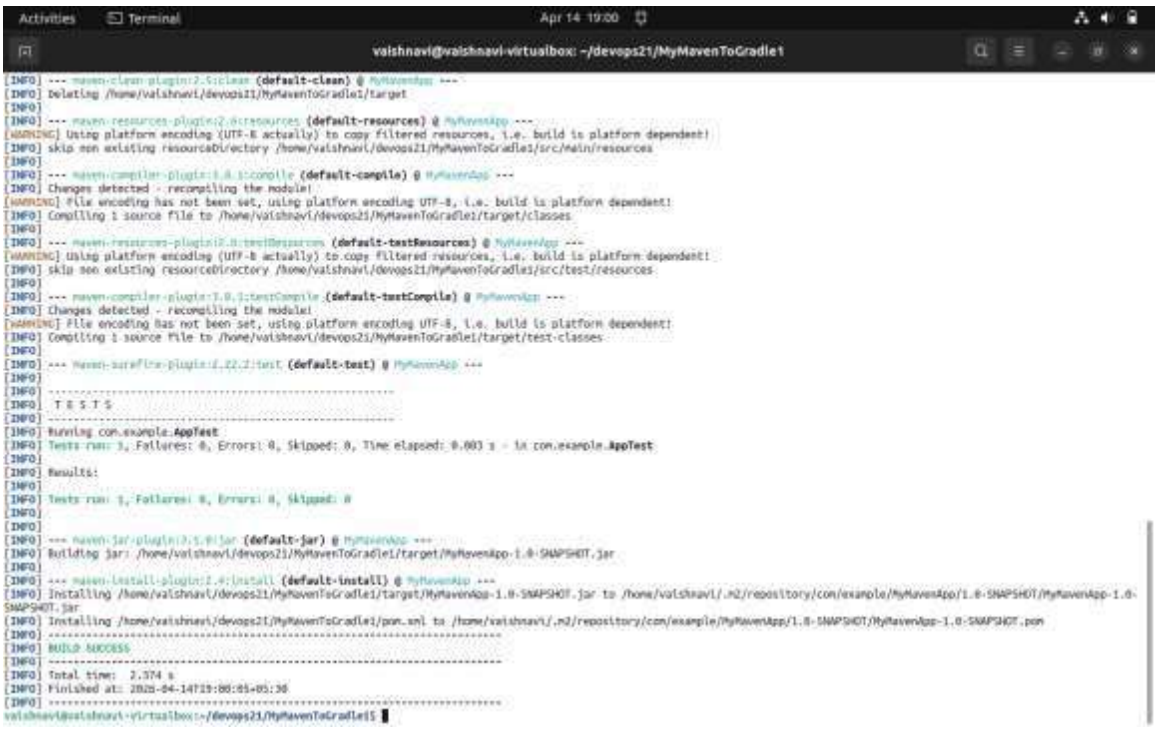
```
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenApp</name>
  <url>http://maven.apache.org</url>

  <!-- Properties: Customize Java version or plugin versions -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
```

```
<version>2.22.2</version>
</plugin>
<plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
<archive>
            <manifestEntries>
               <Main-Class>com.example.MainClass</Main-Class>
            </manifestEntries>
</archive>
        </configuration>
      </plugin>

</plugins>
</build>
</project>
```

Step 3:

Build the project using the command :

$mvn clean install



If the command shoes output as “build success”, it means that the project has compiled successfully

Step 4:

Migrating the maven project to gradle. Execute the commands given below:

$git init –type pom.xml

Error explaination:

- pom.xml is the name of a configuration file used in Maven projects, not a project type. Gradle does not recognize file names in the --type option.
- The --type flag in Gradle expects a valid project type keyword (like java-application, basic, or pom) that tells Gradle what kind of project to create.
- Since pom.xml is not in Gradle’s list of supported types, it throws an error saying it is not supported.On the other hand, pom is a valid type in Gradle, used when you want to create a project compatible with Maven-style structure.

Step 5:

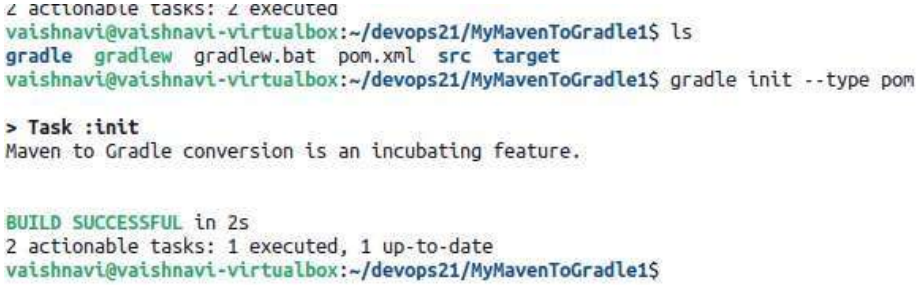
Execute the command to view the files: $ls

There is no build.gradle file in the project.

Step 6:

To correct the error, execute the command:

$git init –type pom



gradle init --type pom is used to initialize (create) a new Gradle project

It tells Gradle to create the project in a Maven-style structure (similar to how Maven projects are organized) It generates

basic files like:

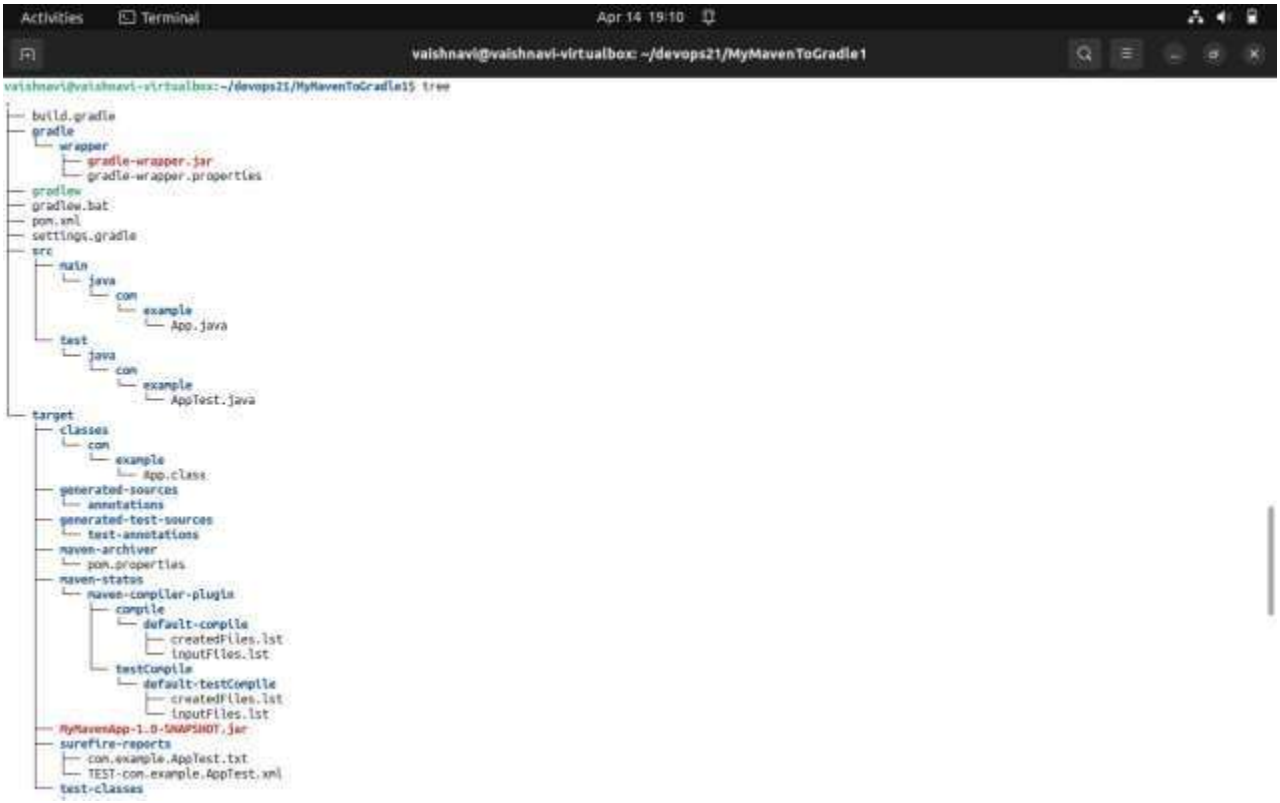
- build.gradle (Gradle build file)
- project folder structure (like src/main/java, etc.)

The pom type specifically is used when you want a simple project setup or a parent project (like Maven's pom packaging)

Step 6:

Execute the same command again or the given command to view the project structure.

$tree



Now, we can make the given observation in the output: This time build.gradle file is created n included in the file

Pushing into the Github

git config --global user.name "vaishnavim1121"

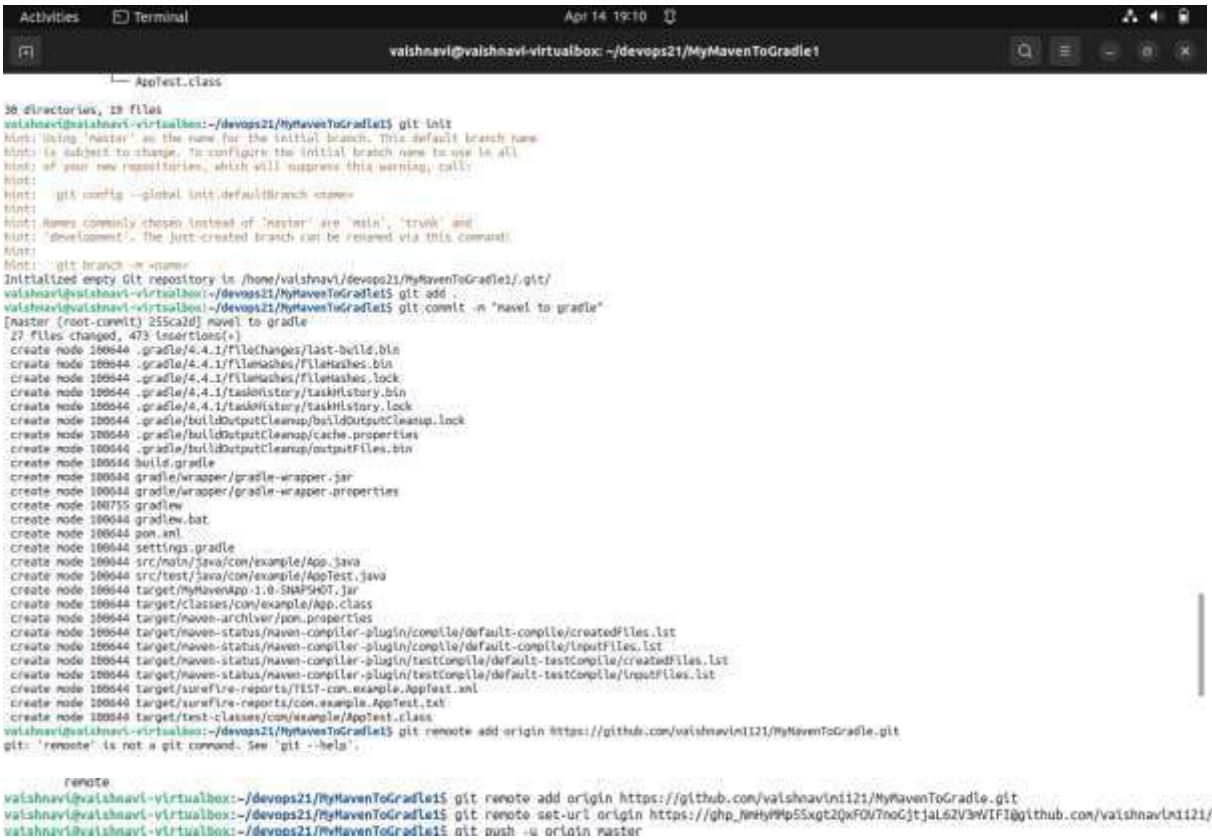
$ git config --global user.email "vaishnavim1121@gmail.com"

$git init $git add . $git commit -m "initial commit"

$ git remote add origin https://github.com/vaishnavim1121/MyMavenToGradle.git

$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenToGradle

$ git push -u origin master




```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$ git push -f origin master
Enumerating objects: 61, done.
Counting objects: 100% (61/61), done.
Delta compression using up to 10 threads
Compressing objects: 100% (34/34), done.
Writing objects: 100% (61/61), 61.01 KiB | 8.72 MiB/s, done.
Total 61 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/vaishnavim1121/MyMavenToGradle
 + 17f5a91...255ca2d master -> master (forced update)
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle1$
```

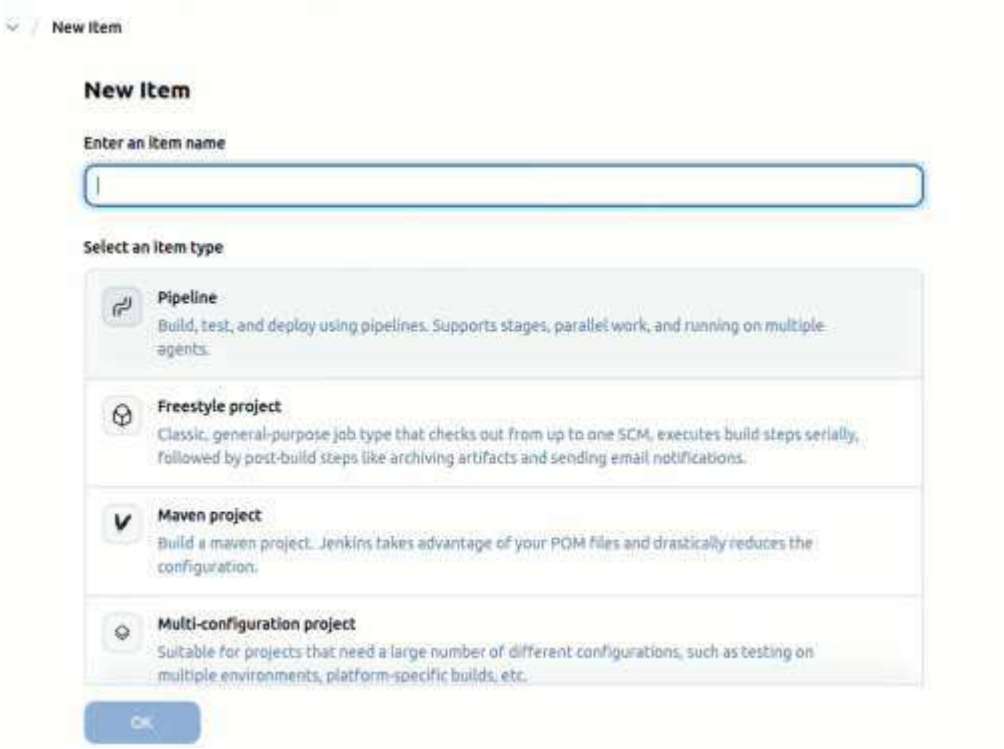
Second Phase

Depicting Jenkins Pipeline

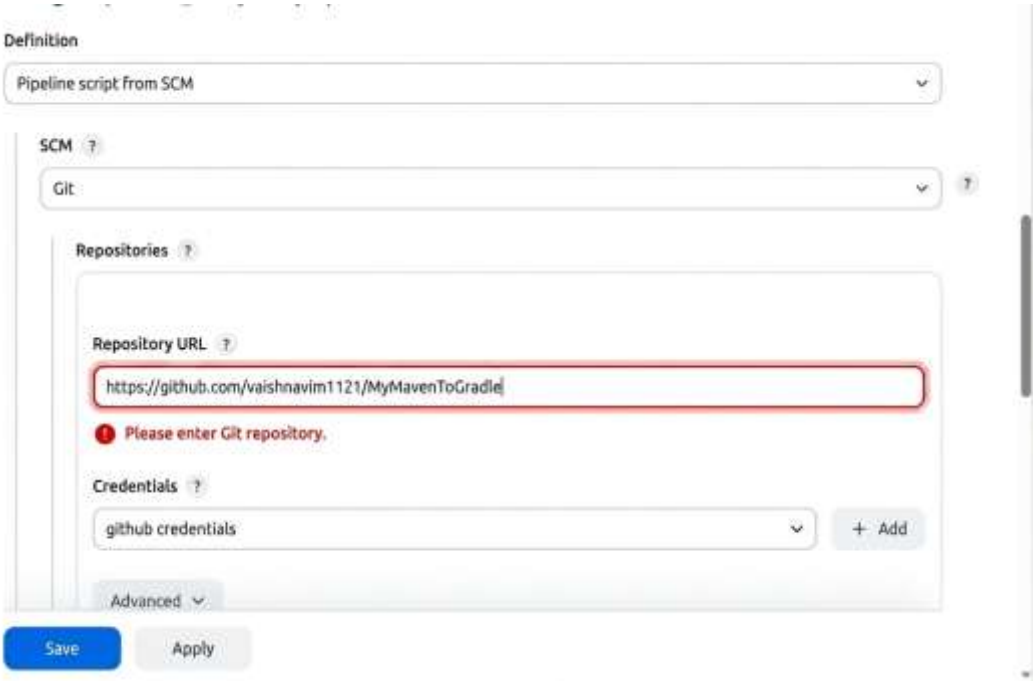
Follow the given steps to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository

go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.



This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click "OK" to proceed to the job configuration page

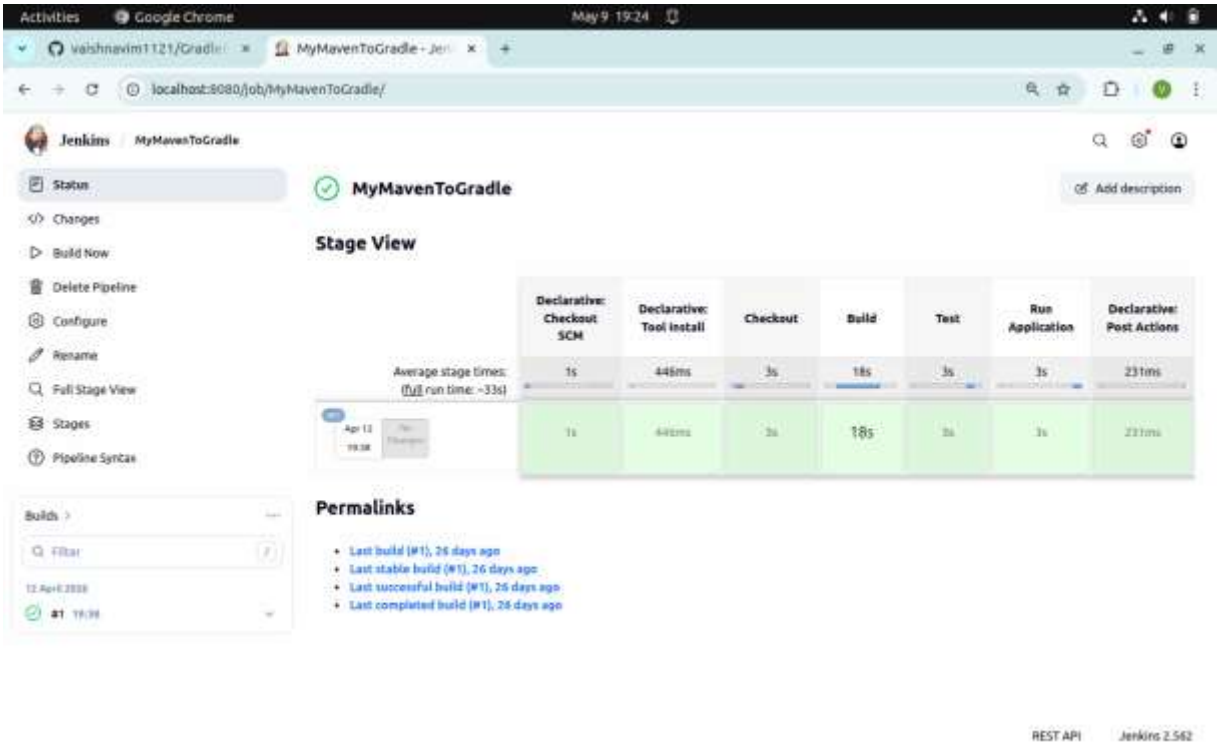


Now we need to add a jenkinsfile into out project which will consist of the operations that will occur when building the project in Jenkins

It includes step wise instructions guiding at each stage with right commands

```
pipeline {
    agent any  // Use any available agent

    tools {
        gradle 'Gradle'  // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/vaishnavim1121/MyGradleApp.git'
            }
        }

        stage('Build') {
            steps {
                sh 'gradle build'  // Run Gradle build
            }
        }

        stage('Test') {
            steps {
                sh 'gradle test'  // Run unit tests
            }
        }




        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'gradle run'
```

Now try building the project with Jenkins



The project is successfully built